

MIAE 215 Final Exam Cheat Sheet on Pointers and Data Types

Course: Mechanical, Industrial & Aerospace engineering (MIAE 215) 378 documents

University: Concordia University

Olivier Gauthier

2024/2025

Download

3

0

Save

↑

i

Arrays
call by reference by default
data type name [# of things]
↓
int, float, char, double, struct, enum
no negative no variables

integer literal
constant integer
X[0] 4 bytes
X[1] 4 bytes
X[2] 4 bytes
X[3] 4 bytes
no gaps between memory of an array

int X[4] = {37, 41, 4, 6}
X[0] = 37 X[2] = 4
X[1] = 41 X[3] = 6
char X[] = "word" (does it for us)

Matrix
int A[3][4] = {
1, 2, 3, 4,
5, 6, 7, 8,
9, 10, 11, 12
};
A[2][3]

Choosing: For (int row=0; row<3; row++)
For (int col=0; col<4; col++)
cin >> A[row][col]

Pointers
int x=10
int* variable name; // ptr only connected to ints
Variable name = &x // ptr references x
int* var.name = 2 // ptr to a ptr
var.name = &variable name
var.name = 30
cout << x << var.name << var.name; gives: 303030

Strings
can be a pointer to anything
void* ptr, with a cast
int x=10;
(int*) ptr = &x;
(int*) ptr = 20;
cout << (int*) ptr << endl;
gives: 2020

Constant Pointers
int x=10;
int* const ptr = &x;
(will always reference x)

New Pointers
int* A;
A = new int(5);
↓
created on "heap space"
initial value 5
Delete A;
A = new int(10);

16 bit converter:
2¹⁶ options
(digital)
high 3.3V
low 0V

10 bit converter (Analog)
1024 options
(0-1023)
analog

File input/output
#include <fstream>
input files
ifstream X;
ofstream Y;
output files
int number;
while (X >> number)
&& number << endl;
takes numbers from X and puts them in while as long as there are numbers

Arduino
#include <servo.h>
void setup(); (initialization)
void loop(); (main loop)
Serial.begin(115200); (bits/sec)
Serial.print("\n text"); (in new line print text)
max 4,300
long unsigned int micros(); (counts time since start)
long unsigned int millis();
void delay(1000); (pauses loop for 1 sec)
Servo motor1; (declares servo)
motor1.attach(7); (sets D7)
motor1.write(90); (turn 90°)

Voltage = $\frac{V}{1023} \cdot 5.0$
Serial.println("temp");
V = analogRead(A0);
(connect SU to A0)
if (analogRead(A0) > 1023)
pinMode(7, OUTPUT);
pinMode(8, INPUT_PULLUP); (default high, low when a push is)
digitalRead(7); (high/low or low/high)
digitalWrite(7, HIGH); (set to high)



<u>structs</u> struct structname { int intname; float floatname; }; structname ting1, ting2; ting1.intname = int value; ting2.floatname = float value; ↑ (dot operator, choose which variable)		<u>C++ cheat sheet</u> User defined functions (outside of int main()) Variable Type Name (int, int*) (includes imported) return ---- ; Library functions (return 0) #include <library function> (using namespace std) calling it: x = name(6, 7)	
<u>Variables</u> (out of range, wraps back) • <u>int</u> : 4 bytes; $(-2^{31}, 2^{31}-1)$ • <u>long</u> : 8 bytes; $(-2^{63}, 2^{63}-1)$ • <u>short</u> : 2 bytes; $(-2^{15}, 2^{15}-1)$ <u>Float</u> : 4 bytes; $(\pm 1.2 \text{E}-38, 3.4 \text{E}38)$ (always signed) (8 digit precision) sign, exponent, mantissa 1 bit 8 bits 23 bits <u>Double</u> : 8 bytes; $(\pm 2.3 \text{E}-308, 1.7 \text{E}308)$ (always signed) (8 digit precision) sign, exponent, mantissa 1 bit 11 bits 52 bits <u>char</u> : 1 byte; $(-128, 127)$ 'A' <u>Bool</u> : 1 byte; (True (1), False (0)) <u>Void</u> : no return type	<u>Relational operators</u> == equality (no float) != inequality x (does not assign, comparison) <u>Ordering operators</u> <, >, <=, >= <u>Logical operators</u> && and or ! not <u>If, else, else if</u> if (true stuff) { // do this; } else if (true stuff) { // do this } else { // if both false, do this }	<u>Loops</u> while (condition true) (For & part 1) { ... } For (int name = 0; name < 5; name++) { ... (does it if 2 is true, do 3 after execution, then checks 2 again) } Do { ... } while (condition true) <u>Call by Value</u> (default) only reads, works as a copy, will not return what we want <u>Call by Reference</u> using reference variable, can work as our value and return it modified Void call by ref (int& b) (ex of function)	<u>cin</u> >> <u>Junk</u> any character at int
<u>cin</u> >> Variable to input <u>cout</u> << "text" & Variable <u>Const Variable</u> Variable_name = Value (needs to be initialized when declared) int & Variable_name = other_variable (have same values now, in memory) it is writeable <u>Enum</u> Variable_name {values, of, name}; (make our own variables with unique values) int % int_name = remainder (7 % 2 = 1) (7 - 2 * 3 = 1) // (comments) end / (skipped)	<u>Switch</u> (only integral types) Switch (Variable type) { case 'char': do something; break; // stops the check or case int: do something; break; }	<u>Static</u> Static int x = 0; x++ → ex: function; x does not reset every time function is called <u>Scope</u> a variable is visible (in scope) in a block {} and its nested sub-blocks (x when there are 2 variables w/ same name, put {} in front of its name to refer to the global)	